

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

I Jabaraj T (192511187) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Design Task

1. Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. AI-Based Fraud Detection Framework

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. Autonomous Supply Chain Optimization System

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

1. HYBRID AI RECOMMENDATION ENGINE DESIGN

Introduction

A Hybrid AI Recommendation Engine combines **Content-Based Filtering** and **Collaborative Filtering** to provide personalized recommendations. Content-based filtering recommends items similar to those previously liked by a user, while collaborative filtering uses the preferences of similar users. PySpark RDDs are used to process large-scale user-item interactions efficiently.

Objectives

- Provide personalized recommendations.
- Process millions of user-item interactions.
- Combine content and collaborative recommendations.
- Adapt recommendations according to user feedback.
- Provide scalable and fault-tolerant processing.

System Architecture

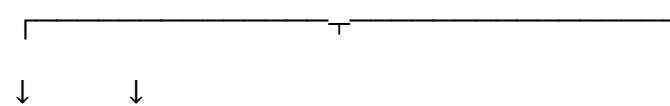
User Interaction Data



Data Collection

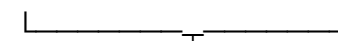


PySpark RDD Processing



Content-Based Collaborative

Filtering Filtering



Hybrid Recommendation Engine



Intelligent Agent



Top-N Recommendations



User Feedback



PySpark and RDD Operations

User interactions such as clicks, ratings and purchases are converted into RDDs.

```
ratings = sc.parallelize(data)
```

```
valid_data = ratings.filter(  
    lambda x: x[2] > 0  
)
```

```
user_items = valid_data.map(  
    lambda x: (x[0], (x[1], x[2]))  
)
```

```
profiles = user_items.groupByKey()
```

Important RDD operations include `map()`, `filter()`, `groupByKey()`, `reduceByKey()` and `join()`.

Hybrid Recommendation

The final recommendation score can be:

$$HybridScore = \alpha(ContentScore) + (1 - \alpha)(CollaborativeScore)$$

For example:

$$HybridScore = 0.4(Content) + 0.6(Collaborative)$$

Intelligent Agent

The intelligent agent continuously monitors:

- Clicks
- Ratings
- Purchases
- Searches
- Likes/dislikes
- Skipped items

Example:

IF user repeatedly views AI courses

THEN increase AI-related recommendation score.

The agent updates the user's profile and changes recommendation weights according to recent behaviour.

Scalability

PySpark divides the user-item dataset into multiple partitions processed by worker nodes in parallel. RDD lineage provides fault tolerance if a partition is lost.

Conclusion

The hybrid system combines distributed data processing, two recommendation techniques and adaptive intelligent agents. It can provide scalable and personalized recommendations for applications such as e-commerce, movies, education and music.

2. SMART DISASTER DETECTION SYSTEM

The question requires a distributed AI system using PySpark and RDDs to detect disasters from **satellite images, sensors and social media**, including data fusion and intelligent-agent coordination.

Introduction

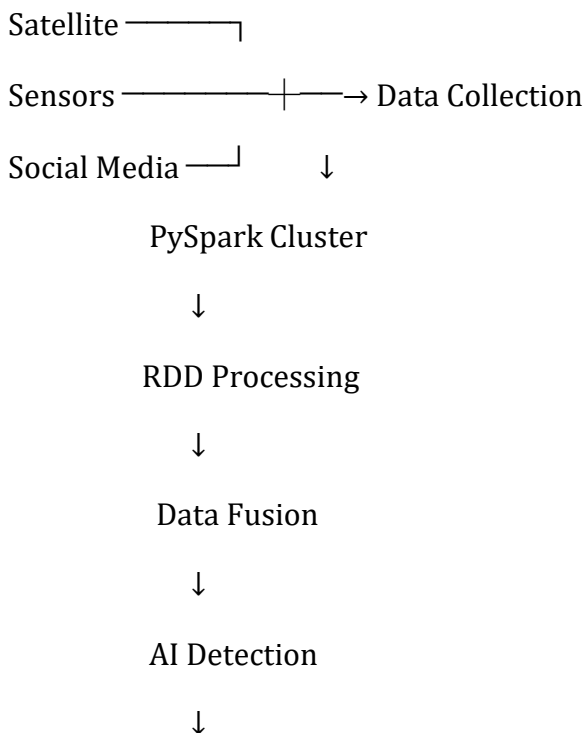
A Smart Disaster Detection System uses distributed AI to detect natural disasters such as floods and earthquakes at an early stage. Information is collected from multiple sources and processed using PySpark.

Data Sources

The proposed system collects:

- Satellite observations
- Rainfall sensors
- Water-level sensors
- Seismic sensors
- Weather information
- GPS data
- Social-media reports

System Architecture



Intelligent Agents



Warning Agent Response Agent



Emergency Alert

PySpark and RDD Processing

Large amounts of sensor information can be represented as RDDs.

```
sensor_rdd = sc.parallelize(sensor_data)
```

```
valid_data = sensor_rdd.filter(  
    lambda x: x["value"] is not None  
)
```

```
region_data = valid_data.map(  
    lambda x: (x["region"], x["value"])  
)
```

```
regional_values = region_data.groupByKey()
```

RDD operations distribute processing across multiple computing nodes.

Data Fusion

Data fusion combines multiple sources to improve reliability.

For example:

Heavy rainfall

+

High river level

+

Flood-related satellite observation

+

Multiple social reports



HIGH FLOOD PROBABILITY

This is more reliable than depending on one sensor.

Intelligent Agents

The system can contain:

Sensor Agent: Collects and validates sensor data.

Detection Agent: Detects abnormal patterns.

Decision Agent: Determines the severity of the disaster.

Communication Agent: Sends alerts.

Response Agent: Recommends evacuation and emergency-response strategies.

Example rule:

IF water level > critical level

AND rainfall = very high

THEN issue flood warning.

Scalability and Fault Tolerance

PySpark processes data from thousands of sensors simultaneously. RDD partitioning provides parallelism, while lineage allows lost data partitions to be recomputed.

Innovation

A confidence score can be calculated from multiple sources:

$$RiskScore = w_1S + w_2W + w_3M$$

where:

- S = Sensor evidence
- W = Weather/satellite evidence
- M = Social-media evidence

Alerts are generated only when sufficient evidence exists.

Conclusion

The proposed system combines PySpark, multi-source data fusion and intelligent agents to provide fast and reliable disaster detection, early warnings and emergency-response support.

3. AI-BASED FRAUD DETECTION FRAMEWORK

The assessment asks for a scalable fraud-detection system using PySpark RDDs, distributed anomaly detection and collaborating intelligent agents.

Introduction

Financial organisations process millions of transactions every day. Manually identifying fraudulent transactions is difficult. An AI-Based Fraud Detection Framework can analyse transaction patterns and detect suspicious activities automatically.

Input Data

The system analyses:

- Transaction amount
- Account ID
- Location
- Device
- Transaction time
- Merchant information
- Transaction frequency
- Previous behaviour

System Architecture

Financial Transactions



Data Ingestion



PySpark Cluster



RDD Processing



Feature Extraction



Anomaly Detection Model



Intelligent Agents



Risk Agent Alert Agent



Fraud Probability



Approve / Review / Block

PySpark RDD Processing

```
transactions = sc.parallelize(data)
```

```
valid = transactions.filter(  
    lambda x: x["amount"] > 0  
)
```

```
account_transactions = valid.map(  
    lambda x: (x["account"], x["amount"]))  
)
```

```
total_amount = account_transactions.reduceByKey(  
    lambda a, b: a + b  
)
```

PySpark distributes transactions across worker nodes, allowing large transaction datasets to be analysed simultaneously.

Anomaly Detection

A transaction is considered suspicious when its behaviour differs greatly from normal customer behaviour.

Example:

Normal behaviour:

₹500 – ₹5,000 transactions in Chennai

New transaction:

₹90,000 transaction from unknown device/location

↓

High anomaly score

Possible features include unusual amount, location, frequency, device and time.

Intelligent Agents

Monitoring Agent: Observes transactions.

Anomaly Agent: Calculates fraud risk.

Decision Agent: Determines whether to approve, review or block.

Alert Agent: Notifies the bank/customer.

Learning Agent: Uses confirmed fraud cases as feedback.

Example:

IF amount is unusually high

AND device is unknown

AND location is unusual

THEN risk = HIGH.

Continuous Improvement

When the bank confirms whether a transaction was fraudulent, the feedback is used to improve future detection.

Transaction → Prediction → Verification

↓

Feedback

↓

Model Improvement

Scalability

RDD partitioning allows transactions to be processed in parallel. Failed partitions can be recomputed using lineage, improving fault tolerance.

Conclusion

The proposed framework combines distributed anomaly detection with intelligent agents to identify fraud quickly. It is scalable, adaptive and suitable for banks, payment applications and other financial systems.

4. AUTONOMOUS SUPPLY CHAIN OPTIMIZATION SYSTEM

The fourth task requires a distributed PySpark AI system for **inventory, logistics and demand forecasting**, with RDD transformations and multiple agents making decentralized decisions.

Introduction

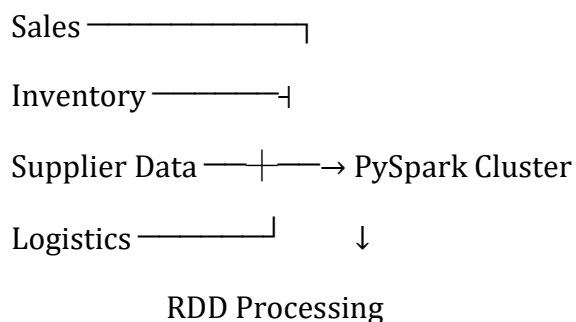
Supply chains generate large amounts of information related to inventory, warehouses, transportation, suppliers and customer demand.

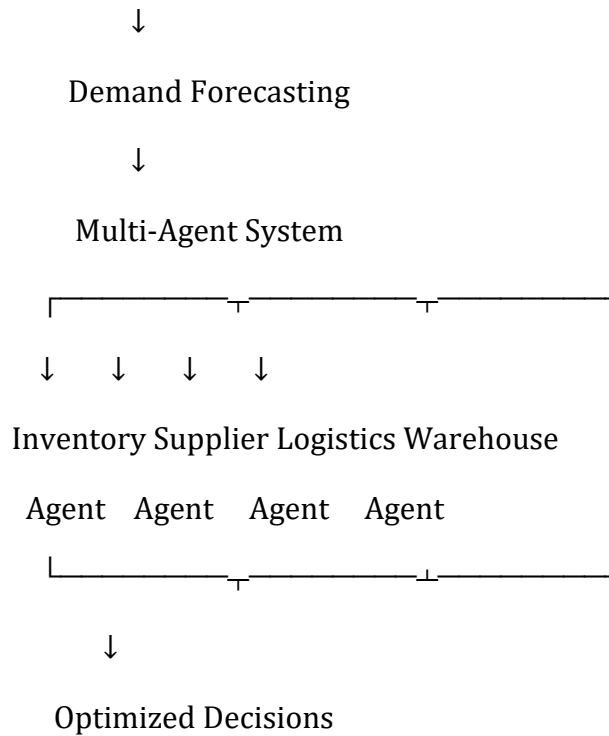
An Autonomous Supply Chain Optimization System uses PySpark and intelligent agents to analyse this data and make efficient decisions.

Objectives

- Prevent stock shortages.
- Reduce excess inventory.
- Forecast future demand.
- Optimize transportation.
- Reduce operational cost.
- Improve delivery time.

Architecture





PySpark RDD Operations

```
sales_rdd = sc.parallelize(sales)
```

```
product_sales = sales_rdd.map(  
    lambda x: (x["product"], x["quantity"])  
)
```

```
total_sales = product_sales.reduceByKey(  
    lambda a, b: a + b  
)
```

map() transforms records, filter() removes invalid records, reduceByKey() aggregates sales, and join() can combine inventory and demand data.

Demand Forecasting

Historical sales information is analysed to predict future demand.

Example:

Historical Sales



RDD Processing



Demand Forecast



Required Inventory

If demand for a product is predicted to increase, inventory can be replenished before a shortage occurs.

Multi-Agent System

Inventory Agent: Maintains appropriate stock levels.

Supplier Agent: Selects suppliers and places orders.

Logistics Agent: Determines efficient delivery routes.

Warehouse Agent: Manages warehouse capacity.

Demand Agent: Predicts future demand.

Example communication:

Demand Agent:

"Product A demand will increase."



Inventory Agent:

"Current stock is insufficient."



Supplier Agent:

"Order additional units."



Logistics Agent:

"Select fastest economical route."

Cost Optimization

The system attempts to minimize:

$$TotalCost = InventoryCost + TransportCost + ShortageCost$$

Agents collaborate to select decisions producing lower total cost.

Scalability

Large datasets from multiple warehouses and stores are partitioned and processed in parallel using PySpark. RDD fault tolerance ensures processing can recover if a worker fails.

Conclusion

The proposed system combines distributed analytics and multiple intelligent agents to automate demand forecasting, inventory management and logistics decisions. This can reduce costs and improve supply-chain efficiency.

5. PERSONALIZED LEARNING ANALYTICS PLATFORM

The final task asks for an AI educational platform using PySpark/RDDs to analyse student behaviour and provide **adaptive learning, intelligent-agent feedback and scalable analytics for thousands of learners.**

Introduction

Students have different learning speeds, strengths and weaknesses. Providing identical learning material to every student may therefore be ineffective.

A Personalized Learning Analytics Platform analyses student behaviour and automatically adapts learning materials according to individual performance.

Input Data

The platform collects:

- Quiz marks
- Assignment scores
- Attendance
- Learning time
- Video completion
- Topics completed
- Number of attempts
- Student feedback

System Architecture

Student Activities



Learning Platform



PySpark Cluster



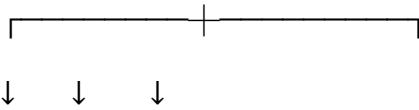
RDD Processing



Learning Analytics

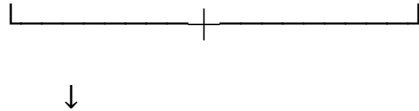


Intelligent Agents



Profile Content Feedback

Agent Agent Agent



Personalized Content



Student



Feedback



PySpark RDD Processing

```
student_rdd = sc.parallelize(student_data)
```

```
valid_scores = student_rdd.filter(
```

```
    lambda x: x["score"] >= 0
```

```
)
```

```
student_scores = valid_scores.map(  
    lambda x: (x["student_id"], (x["score"], 1))  
)
```

```
totals = student_scores.reduceByKey(  
    lambda a, b: (a[0] + b[0], a[1] + b[1])  
)
```

PySpark allows records from thousands of students to be processed simultaneously.

Intelligent Agents

Student Profile Agent: Maintains individual performance profiles.

Assessment Agent: Analyses quiz and assignment results.

Content Agent: Selects appropriate learning materials.

Feedback Agent: Gives immediate feedback.

Learning Agent: Improves recommendations based on student progress.

Adaptive Learning Logic

IF student score < 50%

THEN recommend basic material + practice questions.

IF student score is 50–80%

THEN recommend intermediate content.

IF student score > 80%

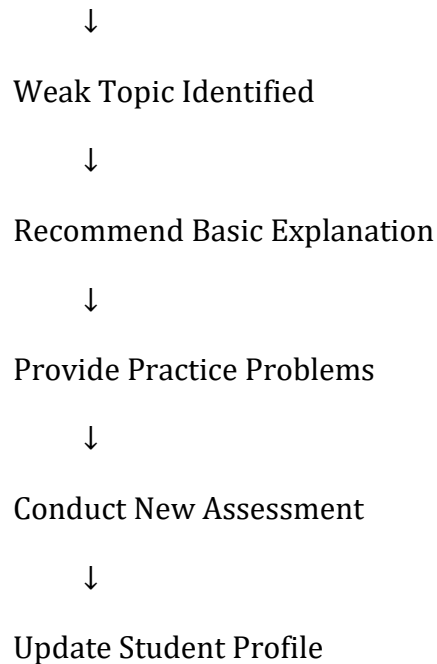
THEN recommend advanced content.

Real-Time Feedback

Suppose a student repeatedly answers questions about **recursion** incorrectly.

The system detects:

Repeated errors in recursion



Thus, learning content changes according to actual student performance.

Scalability

PySpark distributes student data across multiple worker nodes. This makes the platform capable of analysing thousands or millions of learning events.

RDDs also provide fault tolerance through lineage and support efficient parallel transformations.

Innovation

The system can maintain a **dynamic difficulty level**:

$$Difficulty_{new} = Difficulty_{current} + PerformanceAdjustment$$

Students who consistently perform well receive more challenging material, while struggling students receive additional explanations and simpler exercises.

Conclusion

The Personalized Learning Analytics Platform combines PySpark distributed processing with intelligent learning agents to analyse student performance and provide adaptive content. The system can deliver personalized education at scale while continuously improving recommendations using student feedback.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation